



《人工智能与Python程序设计》— 面向对象编程（二）



人工智能与Python程序设计 教研组



回顾



人工智能与
Python程序设计

- 1. 类和实例
- 2. 数据封装
- 3. 访问限制
- 4. 继承和多态



提纲



人工智能与
Python程序设计

- 1. 获取对象信息
- 2. 实例属性和类属性
- 3. 实现二维列表类



获取对象信息

- 获取对象类型与方法
 - 判断对象类型，使用type()函数

```
print(type(123))
print(type('123'))
print(type([1,2,3]))

class Animal(object):
    def run(self):
        print('Animal is running ...')

a = Animal()
print(type(a))
```

```
<class 'int'>
<class 'str'>
<class 'list'>
<class '__main__.Animal'>
```




获取对象信息

- 获取对象类型与方法
 - 判断对象类型，使用type()函数

```
print(type(123))
print(type('123'))
print(type([1,2,3]))

class Animal(object):
    def run(self):
        print('Animal is running ...')

a = Animal()
print(type(a))

<class 'int'>
<class 'str'>
<class 'list'>
<class '__main__.Animal'>
```

```
class Dog(Animal):

    def run(self):
        print('Dog is running...')

    def eat(self):
        print('Eating meat...')

b = Dog()
print(type(b))

<class '__main__.Dog'>
```

父类与子类的class差异



获取对象信息

- 获取对象类型与方法

- 判断对象类型，使用type()函数
- 针对class的继承关系，使用isinstance()函数判断class类型。

例如继承关系：object -> Animal -> Dog -> Husky

```
a = Animal()  
d = Dog()  
h = Husky()
```

```
print(isinstance(h, Husky))  
print(isinstance(h, Dog))  
print(isinstance(h, Animal))
```

```
True  
True  
True
```

```
print(isinstance(d, Husky))  
print(isinstance(h, Dog))  
print(isinstance(d, Animal))
```

```
False  
True  
True
```

获取对象信息

- 获取对象类型与方法
 - 判断对象类型，使用type()函数
 - 针对class的继承关系，使用isinstance()函数判断class类型。
 - 使用dir()来获得一个对象的所有属性和方法，并以一个字符串列表返回

```
a = Animal()  
d = Dog()  
h = Husky()
```

```
print(dir(d))
```

```
['_class_', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__',  
 '__format__', '__ge__', '__getattr__', '__gt__', '__hash__',  
 '__init__', '__init_subclass__', '__le__', '__lt__', '__module__',  
 '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__',  
 '__setattr__', '__sizeof__', '__str__', '__subclasshook__', '__weakref__',  
 'ef_', 'eat', 'run']
```

```
print(d.__class__)
```

```
<class '__main__.Dog'>
```


获取对象信息

- 获取对象类型与方法
 - 判断对象类型，使用type()函数
 - 针对class的继承关系，使用isinstance()函数判断class类型。
 - 使用dir()来获得一个对象的所有属性和方法，并以一个字符串列表返回

```
a = Animal()  
d = Dog()  
h = Husky()
```

```
print(dir(d))  
  
['_class_', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__',  
'_format_', '__ge__', '__getattribute__', '__gt__', '__hash__',  
'_init_', '__init_subclass__', '__le__', '__lt__', '__module__',  
'_ne_', '__new__', '__reduce__', '__reduce_ex__', '__repr__',  
'_setattr__', '__sizeof__', '__str__', '__subclasshook__', '__weakref__',  
'eat', 'run']
```

```
print(d.__class__)  
  
<class '__main__.Dog'>
```

类似__xxx__的属性和方法在Python中都是有特殊用途的，比如__class__方法返回所属类。

获取对象信息

- 获取对象类型与方法
 - 判断对象类型，使用type()函数
 - 针对class的继承关系，使用isinstance()函数判断class类型。
 - 使用dir()来获得一个对象的所有属性和方法，并以一个字符串列表返回

```
a = Animal()  
d = Dog()  
h = Husky()
```

```
print(dir(d))  
  
['_class_', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__',  
'_format_', '__ge__', '__getattribute__', '__gt__', '__hash__',  
'_init_', '__init_subclass__', '__le__', '__lt__', '__module__',  
'_ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__',  
'_setattr__', '__sizeof__', '__str__', '__subclasshook__', '__weakref__',  
'eat', 'run']
```

```
print(d.__class__)  
  
<class '__main__.Dog'>
```

dir()也会返回类内的方法

获取对象信息

- 获取对象类型与方法
 - 判断对象类型, 使用type()函数
 - 针对class的继承关系, 使用isinstance()函数判断class类型。
 - 使用dir()来获得一个对象的所有属性和方法, 并以一个字符串列表返回



```
dir([1,2,3])
['__add__', '__class__', '__contains__', '__delattr__', '__delitem__', '__delslice__', '__eq__', '__format__', '__ge__', '__getattribute__', '__getitem__', '__iadd__', '__iadd__', '__imul__', '__init__', '__iter__', '__le__', '__len__', '__list__', '__lt__', '__mod__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__reversed__', '__rmul__', '__setattr__', '__setitem__', '__setslice__', '__sizeof__', '__str__', '__subclasshook__', 'append', 'clear', 'copy', 'count', 'extend', 'index', 'insert', 'pop', 'remove', 'reverse', 'sort']
```

```
['append',
'clear',
'copy',
'count',
'extend',
'index',
'insert',
'pop',
'remove',
'reverse',
'sort']
```

dir()亦可用于Python内置数据类型



获取对象信息

- 获取对象类型与方法
 - 判断对象类型，使用type()函数
 - 针对class的继承关系，使用isinstance()函数判断class类型。
 - 使用dir()来获得一个对象的所有属性和方法，并以一个字符串列表返回
 - 仅仅把属性和方法列出来是不够的，配合getattr()、setattr()以及hasattr()，可直接操作一个对象的状态

```
class Animal(object):  
    def __init__(self):  
        self.able_to_move = True  
    def run(self):  
        print('Animal is running...')
```

```
a = Animal()  
print(hasattr(a, 'able_to_move'))  
setattr(a, 'able_to_move', False)  
print(getattr(a, 'able_to_move'))
```

```
True  
False
```




获取对象信息

- 注意：
 - 通过内置的一系列函数，我们可以对任意一个Python对象进行剖析，拿到其内部的数据。要注意的是，只有在不知道对象信息的时候，我们才会去获取对象信息。

如果可以直写

```
sum = obj.x + obj.y
```

就不要写

```
sum = getattr(obj, 'x') + getattr(obj, 'y')
```



提纲



人工智能与
Python程序设计

- 1. 获取对象信息
- 2. 实例属性和类属性
- 3. 实现二维列表类

实例属性与类属性

- 由于Python是动态语言，根据类创建的实例可以任意绑定属性。
 - 给实例绑定属性的方法是通过实例变量，或者通过self变量

```
class Student(object):  
    def init (self, name):  
        self.name = name  
  
s = Student('Bob')  
s.score = 90
```




实例属性与类属性

- 由于Python是动态语言，根据类创建的实例可以任意绑定属性。
 - 给实例绑定属性的方法是通过实例变量，或者通过self变量
- 给类本身绑定属性，可以直接在class中定义属性，称为类属性，归类所有

```
class Student(object):  
    stu_num = 396  
  
    def __init__(self, name):  
        self.name = name
```



实例属性与类属性

- 由于Python是动态语言，根据类创建的实例可以任意绑定属性。
 - 给实例绑定属性的方法是通过实例变量，或者通过self变量
- 给类本身绑定属性，可以直接在class中定义属性，称为类属性，归类所有

```
class Student(object):  
    stu_num = 396  
  
    def __init__(self, name):  
        self.name = name  
  
s = Student('Bob')  
print(s.name)  
print(s.stu_num)
```

```
Bob  
396
```

定义了一个类属性后，这个属性虽然归类所有，但类的所有实例都可以访问到。



实例属性与类属性

- 注意：
 - 在编写程序的时候，不要对实例属性和类属性使用相同的名字，因为相同名称的实例属性将屏蔽掉类属性

```
class Student(object):  
    name = 'student'  
  
    def __init__(self, name):  
        self.name = name  
  
s = Student('Bob')  
print(s.name)
```

Bob



提纲



人工智能与
Python程序设计

- 1. 获取对象信息
- 2. 实例属性和类属性
- 3. 实现二维列表类

实现二维列表类

- 二维列表
 - 二维列表的初始化
 - 二维列表的访问，索引和获取
 - 二维列表的元素增加与删除

```
2 class my2DList(object):
3
4     def __init__(self, size=None, defaultValues=0):...
44
45     def is2DList(self, data):...
66
67     def getMy2DList(self):...
72
73     def myLen(self):...
78
79     def myIndex(self, rowIndex=None, colIndex=None):...
115
116     def myAppendRow(self, rowData):...
126
127     def myAppendCol(self, colData):...
138
139     def myDeleteRow(self, index):...
153
154     def myDeleteCol(self, index):...
```



实现二维列表类

- 二维列表
 - 二维列表的初始化（构造函数）
 - 从给定二维列表初始化
 - 指定列表大小，所有元素赋值相同
 - 从给定的一维列表初始化二维列表，并根据参数定义列表大小

实现二维列表类

- 二维列表
 - 二维列表的初始化（构造函数）

```
def __init__(self, size=None, defaultValues = 0):  
    ...  
    二维列表类的构造函数  
    参数self: 创建的实例本身  
    参数size: 指定二维列表大小  
    参数defaultValues: 输入指定列表  
    ...  
  
    if (size is None) and (not self.is2DList(defaultValues)):  
        # 未指定二维列表大小, 且指定列表不是二维列表  
        print('Please input proper 2D list!')  
  
    elif (size is None) and self.is2DList(defaultValues):  
        # 未指定二维列表大小, 但指定列表是二维列表  
        self.__items = defaultValues  
        self.__size = [len(defaultValues), len(defaultValues[0])]  
  
    else:  
        # 根据指定二维列表大小, 创建二维列表  
        self.__size = list(size)  
        if(isinstance(defaultValues, int) or isinstance(defaultValues, float)):  
            # 利用defaultValues作为二维列表元素  
            self.__items = list()  
            for i in range(self.__size[0]):  
                subItems = list()  
                for j in range(self.__size[1]):  
                    subItems.append(defaultValues)  
                self.__items.append(subItems)  
  
            # 利用defaultValues指定的一维列表初始化二维列表  
        elif(isinstance(defaultValues, list) and len(defaultValues) == self.__size[0]*self.__size[1]):  
            self.__items = list()  
            for i in range(self.__size[0]):  
                subItems = list()  
                for j in range(self.__size[1]):  
                    subItems.append(defaultValues[i*self.__size[1]+j])  
                self.__items.append(subItems)  
  
    else:  
        print('Incorrect defaultValues')
```

实现二维列表类

- 二维列表
 - 二维列表的初始化（构造函数）
 - 输入参数判定是否合法

```
def __init__(self, size=None, defaultValues = 0):  
    ...  
    二维列表类的构造函数  
    参数self: 创建的实例本身  
    参数size: 指定二维列表大小  
    参数defaultValues: 输入指定列表  
    ...  
  
    if (size is None) and (not self.is2DList(defaultValues)):  
        # 未指定二维列表大小, 且指定列表不是二维列表  
        print('Please input proper 2D list!')  
  
    elif (size is None) and self.is2DList(defaultValues):  
        # 未指定二维列表大小, 但指定列表是二维列表  
        self.__items = defaultValues  
        self.__size = [len(defaultValues), len(defaultValues[0])]  
  
    else:  
        # 根据指定二维列表大小, 创建二维列表  
        self.__size = list(size)  
        if(isinstance(defaultValues, int) or isinstance(defaultValues, float)):  
            # 利用defaultValues作为二维列表元素  
            self.__items = list()  
            for i in range(self.__size[0]):  
                subItems = list()  
                for j in range(self.__size[1]):  
                    subItems.append(defaultValues)  
                self.__items.append(subItems)  
  
            # 利用defaultValues指定的一维列表初始化二维列表  
            elif(isinstance(defaultValues, list) and len(defaultValues) == self.__size[0]*self.__size[1]):  
                self.__items = list()  
                for i in range(self.__size[0]):  
                    subItems = list()  
                    for j in range(self.__size[1]):  
                        subItems.append(defaultValues[i*self.__size[1]+j])  
                    self.__items.append(subItems)  
  
        else:  
            print('Incorrect defaultValues')
```



实现二维列表类

- 二维列表
 - 二维列表的初始化（构造函数）
 - 输入参数判定是否合法

```
def is2DList(self, data):  
    '''  
    类的方法：判断data是否为二维列表  
    '''  
  
    if not isinstance(data, list):  
        # 判断data的第一维是否为list数据类型  
        return False  
    else:  
        colLen = len(data[0])  
        for row in data:  
            if (not isinstance(row, list)) or (len(row) != colLen):  
                # 判断data的第二维是否为list数据类型，且元素数目是否相等  
                return False  
            else:  
                for i in range(colLen):  
                    if ((not isinstance(row[i], int)) and (not isinstance(row[i], float))):  
                        # 判断data的第二维内元素是否为int类型或者float类型  
                        return False  
        return True # 以上均符合要求，返回True
```

```
def __init__(self, size=None, defaultValues = 0):  
    '''  
    二维列表类的构造函数  
    参数self: 创建的实例本身  
    参数size: 指定二维列表大小  
    参数defaultValues: 输入指定列表  
    '''  
  
    if (size is None) and (not self.is2DList(defaultValues)):  
        # 未指定二维列表大小，且指定列表不是二维列表  
        print('Please input proper 2D list!')  
  
    elif (size is None) and self.is2DList(defaultValues):  
        # 未指定二维列表大小，但指定列表是二维列表  
        self.__items = defaultValues  
        self.__size = [len(defaultValues), len(defaultValues[0])]   
  
    else:  
        # 根据指定二维列表大小，创建二维列表  
        self.__size = list(size)  
        if(isinstance(defaultValues, int) or isinstance(defaultValues, float)):  
            # 利用defaultValues作为二维列表元素  
            self.__items = list()  
            for i in range(self.__size[0]):  
                subItems = list()  
                for j in range(self.__size[1]):  
                    subItems.append(defaultValues)  
                self.__items.append(subItems)  
  
            # 利用defaultValues指定的一维列表初始化二维列表  
        elif(isinstance(defaultValues, list) and len(defaultValues) == self.__size[0]*self.__size[1]):  
            self.__items = list()  
            for i in range(self.__size[0]):  
                subItems = list()  
                for j in range(self.__size[1]):  
                    subItems.append(defaultValues[i*self.__size[1]+j])  
                self.__items.append(subItems)  
  
    else:  
        print('Incorrect defaultValues')
```


实现二维列表类

- 二维列表
 - 二维列表的初始化 (构造函数)
 - 从给定二维列表初始化

```
def __init__(self, size=None, defaultValues = 0):  
    ...  
    二维列表类的构造函数  
    参数self: 创建的实例本身  
    参数size: 指定二维列表大小  
    参数defaultValues: 输入指定列表  
    ...  
  
    if (size is None) and (not self.is2DList(defaultValues)):  
        # 未指定二维列表大小, 且指定列表不是二维列表  
        print('Please input proper 2D list!')  
  
    elif (size is None) and self.is2DList(defaultValues):  
        # 未指定二维列表大小, 但指定列表是二维列表  
        self.__items = defaultValues  
        self.__size = [len(defaultValues), len(defaultValues[0])]   
  
    else:  
        # 根据指定二维列表大小, 创建二维列表  
        self.__size = list(size)  
        if(isinstance(defaultValues, int) or isinstance(defaultValues, float)):  
            # 利用defaultValues作为二维列表元素  
            self.__items = list()  
            for i in range(self.__size[0]):  
                subItems = list()  
                for j in range(self.__size[1]):  
                    subItems.append(defaultValues)  
                self.__items.append(subItems)  
  
            # 利用defaultValues指定的一维列表初始化二维列表  
        elif(isinstance(defaultValues, list) and len(defaultValues) == self.__size[0]*self.__size[1]):  
            self.__items = list()  
            for i in range(self.__size[0]):  
                subItems = list()  
                for j in range(self.__size[1]):  
                    subItems.append(defaultValues[i*self.__size[1]+j])  
                self.__items.append(subItems)  
  
        else:  
            print('Incorrect defaultValues')
```

实现二维列表类

- 二维列表
 - 二维列表的初始化 (构造函数)
 - 指定列表大小, 元素赋值相同

```
def __init__(self, size=None, defaultValues = 0):  
    ...  
    二维列表类的构造函数  
    参数self: 创建的实例本身  
    参数size: 指定二维列表大小  
    参数defaultValues: 输入指定列表  
    ...  
  
    if (size is None) and (not self.is2DList(defaultValues)):  
        # 未指定二维列表大小, 且指定列表不是二维列表  
        print('Please input proper 2D list!')  
  
    elif (size is None) and self.is2DList(defaultValues):  
        # 未指定二维列表大小, 但指定列表是二维列表  
        self.__items = defaultValues  
        self.__size = [len(defaultValues), len(defaultValues[0])]  
  
    else:  
        # 根据指定二维列表大小, 创建二维列表  
        self.__size = list(size)  
        if(isinstance(defaultValues, int) or isinstance(defaultValues, float)):  
            # 利用defaultValues作为二维列表元素  
            self.__items = list()  
            for i in range(self.__size[0]):  
                subItems = list()  
                for j in range(self.__size[1]):  
                    subItems.append(defaultValues)  
                self.__items.append(subItems)  
  
        # 利用defaultValues指定的一维列表初始化二维列表  
        elif(isinstance(defaultValues, list) and len(defaultValues) == self.__size[0]*self.__size[1]):  
            self.__items = list()  
            for i in range(self.__size[0]):  
                subItems = list()  
                for j in range(self.__size[1]):  
                    subItems.append(defaultValues[i*self.__size[1]+j])  
                self.__items.append(subItems)  
  
    else:  
        print('Incorrect defaultValues')
```

实现二维列表类

- 二维列表
 - 二维列表的初始化 (构造函数)
 - 从一维列表初始化二维列表

```
def __init__(self, size=None, defaultValues = 0):  
    """  
    二维列表类的构造函数  
    参数self: 创建的实例本身  
    参数size: 指定二维列表大小  
    参数defaultValues: 输入指定列表  
    """  
  
    if (size is None) and (not self.is2DList(defaultValues)):  
        # 未指定二维列表大小, 且指定列表不是二维列表  
        print('Please input proper 2D list!')  
  
    elif (size is None) and self.is2DList(defaultValues):  
        # 未指定二维列表大小, 但指定列表是二维列表  
        self.__items = defaultValues  
        self.__size = [len(defaultValues), len(defaultValues[0])]  
  
    else:  
        # 根据指定二维列表大小, 创建二维列表  
        self.__size = list(size)  
        if(isinstance(defaultValues, int) or isinstance(defaultValues, float)):  
            # 利用defaultValues作为二维列表元素  
            self.__items = list()  
            for i in range(self.__size[0]):  
                subItems = list()  
                for j in range(self.__size[1]):  
                    subItems.append(defaultValues)  
                self.__items.append(subItems)  
  
            # 利用defaultValues指定的一维列表初始化二维列表  
            elif(isinstance(defaultValues, list) and len(defaultValues) == self.__size[0]*self.__size[1]):  
                self.__items = list()  
                for i in range(self.__size[0]):  
                    subItems = list()  
                    for j in range(self.__size[1]):  
                        subItems.append(defaultValues[i*self.__size[1]+j])  
                    self.__items.append(subItems)  
  
        else:  
            print('Incorrect defaultValues')
```




实现二维列表类

- 二维列表
 - 二维列表的访问，索引和获取
 - 二维列表的访问（私有变量访问）

```
def getMy2DList(self):  
    """  
    类的方法：返回二维列表  
    """  
    return self.__items
```



实现二维列表类

- 二维列表
 - 二维列表的访问，索引和获取
 - 二维列表的访问（私有变量访问）

```
def mySize(self):  
    """  
    类的方法：返回二维列表的大小  
    """  
    return self.__size[0], self.__size[1]
```

实现二维列表类

- 二维列表
 - 二维列表的访问，索引和获取
 - 二维列表的索引和获取

```
def myIndex(self, rowIndex=None, colIndex=None):
    """
    类的方法：对二维列表索引
    参数rowIndex：索引的行坐标
    参数colIndex：索引的列坐标
    """
    if (rowIndex is None) and (colIndex is None):
        # rowIndex和colIndex均未给定，输出错误
        print('Please input index number!')

    elif (isinstance(rowIndex, int) and (colIndex is None)):
        # 仅给定行索引值，则仅对行进行索引
        if (rowIndex >= 0) and (rowIndex < self.__size[0]):
            return self.__items[rowIndex]
        else:
            print('Row index out of range!')

    elif (rowIndex is None) and isinstance(colIndex, int):
        # 仅给定列索引值，则仅对列进行索引
        if (colIndex >= 0) and (colIndex < self.__size[1]):
            colItems = list()
            for row in self.__items:
                colItems.append(row[colIndex])
            return colItems
        else:
            print('Column index out of range!')

    elif (isinstance(colIndex, int) and isinstance(rowIndex, int)):
        # 给定行和列索引值，则对指定元素进行索引
        if (rowIndex >= 0) and (rowIndex < self.__size[0]) and (colIndex >= 0) and (colIndex < self.__size[1]):
            return self.__items[rowIndex][colIndex]
        else:
            print('Index out of range!')

    else:
        print('Please input int-type index!')
```


实现二维列表类

- 二维列表
 - 二维列表的访问，索引和获取
 - 二维列表的索引和获取
 - 索引值判定是否合法

```
def myIndex(self, rowIndex=None, colIndex=None):  
    """  
    类的方法：对二维列表索引  
    参数rowIndex：索引的行坐标  
    参数colIndex：索引的列坐标  
    """  
    if (rowIndex is None) and (colIndex is None):  
        # rowIndex和colIndex均未给定，输出错误  
        print('Please input index number!')  
  
    elif (isinstance(rowIndex, int) and (colIndex is None)):  
        # 仅给定行索引值，则仅对行进行索引  
        if (rowIndex >= 0) and (rowIndex < self.__size[0]):  
            return self.__items[rowIndex]  
        else:  
            print('Row index out of range!')  
  
    elif (rowIndex is None) and isinstance(colIndex, int):  
        # 仅给定列索引值，则仅对列进行索引  
        if (colIndex >= 0) and (colIndex < self.__size[1]):  
            colItems = list()  
            for row in self.__items:  
                colItems.append(row[colIndex])  
            return colItems  
        else:  
            print('Column index out of range!')  
  
    elif (isinstance(colIndex, int) and isinstance(rowIndex, int)):  
        # 给定行和列索引值，则对指定元素进行索引  
        if (rowIndex >= 0) and (rowIndex < self.__size[0]) and (colIndex >= 0) and (colIndex < self.__size[1]):  
            return self.__items[rowIndex][colIndex]  
        else:  
            print('Index out of range!')  
  
    else:  
        print('Please input int-type index!')
```



实现二维列表类

- 二维列表
 - 二维列表的访问，索引和获取
 - 二维列表的索引和获取
 - 仅给定行索引值

```
def myIndex(self, rowIndex=None, colIndex=None):
    """
    类的方法：对二维列表索引
    参数rowIndex：索引的行坐标
    参数colIndex：索引的列坐标
    """
    if (rowIndex is None) and (colIndex is None):
        # rowIndex和colIndex均未给定，输出错误
        print('Please input index number!')

    elif (isinstance(rowIndex, int) and (colIndex is None)):
        # 仅给定行索引值，则仅对行进行索引
        if (rowIndex >= 0) and (rowIndex < self.__size[0]):
            return self.__items[rowIndex]
        else:
            print('Row index out of range!')

    elif (rowIndex is None) and isinstance(colIndex, int):
        # 仅给定列索引值，则仅对列进行索引
        if (colIndex >= 0) and (colIndex < self.__size[1]):
            colItems = list()
            for row in self.__items:
                colItems.append(row[colIndex])
            return colItems
        else:
            print('Column index out of range!')

    elif (isinstance(colIndex, int) and isinstance(rowIndex, int)):
        # 给定行和列索引值，则对指定元素进行索引
        if (rowIndex >= 0) and (rowIndex < self.__size[0]) and (colIndex >= 0) and (colIndex < self.__size[1]):
            return self.__items[rowIndex][colIndex]
        else:
            print('Index out of range!')

    else:
        print('Please input int-type index!')
```

实现二维列表类

- 二维列表
 - 二维列表的访问，索引和获取
 - 二维列表的索引和获取
 - 仅给定列索引值

```
def myIndex(self, rowIndex=None, colIndex=None):
    """
    类的方法：对二维列表索引
    参数rowIndex: 索引的行坐标
    参数colIndex: 索引的列坐标
    """
    if (rowIndex is None) and (colIndex is None):
        # rowIndex和colIndex均未给定，输出错误
        print('Please input index number!')

    elif (isinstance(rowIndex, int) and (colIndex is None)):
        # 仅给定行索引值，则仅对行进行索引
        if (rowIndex >= 0) and (rowIndex < self.__size[0]):
            return self.__items[rowIndex]
        else:
            print('Row index out of range!')

    elif (rowIndex is None) and isinstance(colIndex, int):
        # 仅给定列索引值，则仅对列进行索引
        if (colIndex >= 0) and (colIndex < self.__size[1]):
            colItems = list()
            for row in self.__items:
                colItems.append(row[colIndex])
            return colItems
        else:
            print('Column index out of range!')

    elif (isinstance(colIndex, int) and isinstance(rowIndex, int)):
        # 给定行和列索引值，则对指定元素进行索引
        if (rowIndex >= 0) and (rowIndex < self.__size[0]) and (colIndex >= 0) and (colIndex < self.__size[1]):
            return self.__items[rowIndex][colIndex]
        else:
            print('Index out of range!')

    else:
        print('Please input int-type index!')
```


实现二维列表类

- 二维列表
 - 二维列表的访问，索引和获取
 - 二维列表的索引和获取
 - 行、列索引均给定

```
def myIndex(self, rowIndex=None, colIndex=None):
    """
    类的方法：对二维列表索引
    参数rowIndex: 索引的行坐标
    参数colIndex: 索引的列坐标
    """
    if (rowIndex is None) and (colIndex is None):
        # rowIndex和colIndex均未给定，输出错误
        print('Please input index number!')

    elif (isinstance(rowIndex, int) and (colIndex is None)):
        # 仅给定行索引值，则仅对行进行索引
        if (rowIndex >= 0) and (rowIndex < self.__size[0]):
            return self.__items[rowIndex]
        else:
            print('Row index out of range!')

    elif (rowIndex is None) and isinstance(colIndex, int):
        # 仅给定列索引值，则仅对列进行索引
        if (colIndex >= 0) and (colIndex < self.__size[1]):
            colItems = list()
            for row in self.__items:
                colItems.append(row[colIndex])
            return colItems
        else:
            print('Column index out of range!')

    elif (isinstance(colIndex, int) and isinstance(rowIndex, int)):
        # 给定行和列索引值，则对指定元素进行索引
        if (rowIndex >= 0) and (rowIndex < self.__size[0]) and (colIndex >= 0) and (colIndex < self.__size[1]):
            return self.__items[rowIndex][colIndex]
        else:
            print('Index out of range!')

    else:
        print('Please input int-type index!')
```



实现二维列表类

- 二维列表
 - 二维列表的元素**增加**与删除

```
def myAppendRow(self, rowData):  
    """  
    类的方法：向二维列表行末添加新的一行  
    参数rowData: 输入的添加行  
    """  
    if self.__size[1] == len(rowData):  
        self.__items.append(rowData)  
        self.__size[0] += 1  
    else:  
        print('Appended data not fit the List row!')  
  
def myAppendCol(self, colData):  
    """  
    类的方法：向二维列表列末添加新的一列  
    参数colData: 输入的添加列  
    """  
    if self.__size[0] == len(colData):  
        for i in range(self.__size[1]):  
            self.__items[i].append(colData[i])  
        self.__size[1] += 1  
    else:  
        print('Appended data not fit the List column!')
```

实现二维列表类

- 二维列表
 - 二维列表的元素增加与删除

```
def myDeleteRow(self, index):  
    """  
    类的方法：删除二维列表的中指定行  
    参数index：需删除的行索引  
    """  
    if index >= 0 and index < self.__size[0]:  
        self.newItems = list()  
        for i in range(self.__size[0]):  
            if i != index:  
                self.newItems.append(self.__items[i])  
        self.__items = self.newItems  
        self.__size[0] -= 1  
    else:  
        print('Index out of range!!')  
  
def myDeleteCol(self, index):  
    """  
    类的方法：删除二维列表的中指定列  
    参数index：需删除的列索引  
    """  
    if index >= 0 and index < self.__size[1]:  
        self.newItems = list()  
        for i in range(self.__size[0]):  
            subItems = list()  
            for j in range(self.__size[1]):  
                if j != index:  
                    subItems.append(self.__items[i][j])  
            self.newItems.append(subItems)  
        self.__items = self.newItems  
        self.__size[1] -= 1  
    else:  
        print('Index out of range!!')
```


实现二维列表类

- 二维列表
 - 二维列表的初始化（构造函数）
 - 二维列表的访问，索引和获取
 - 二维列表的元素增加与删除

```
2 class my2DList(object):
3
4     def __init__(self, size=None, defaultValues=0):...
44
45     def is2DList(self, data):...
66
67     def getMy2DList(self):...
72
73     def myLen(self):...
78
79     def myIndex(self, rowIndex=None, colIndex=None):...
115
116     def myAppendRow(self, rowData):...
126
127     def myAppendCol(self, colData):...
138
139     def myDeleteRow(self, index):...
153
154     def myDeleteCol(self, index):...
```



实现二维列表类

- 二维列表

- 二维列表的初始化

- 指定列表大小，所有元素赋值相同
 - 从给定的一维列表初始化二维列表，并根据参数定义列表大小
 - 从给定二维列表初始化

```
In [70]: customList0 = my2DList()
         Please input proper 2D list!

In [71]: customList1 = my2DList(size=(2,3), defaultValues=1)

In [72]: customList1.getMy2DList()
Out[72]: [[1, 1, 1], [1, 1, 1]]

In [73]: customList2 = my2DList(size=(2,3), defaultValues=[1,2,3,4,5,6])

In [74]: customList2.getMy2DList()
Out[74]: [[1, 2, 3], [4, 5, 6]]

In [75]: customList3 = my2DList(size=None, defaultValues=[[1,3,6],[4,6,6]])

In [76]: customList3.getMy2DList()
Out[76]: [[1, 3, 6], [4, 6, 6]]
```



实现二维列表类

- 二维列表
 - 二维列表的访问，索引和获取

```
In [81]: customList = my2DList(size=(2,3), defaultValues=[1,2,3,4,5,6]) # 初始化
```

```
Out[81]: (2, 3)
```

```
In [82]: customList.getMy2DList() # 2D list 获取
```

```
Out[82]: [[1, 2, 3], [4, 5, 6]]
```

```
In [83]: customList.mySize() # 2D list size 获取
```

```
Out[83]: (2, 3)
```

```
In [84]: customList.myIndex(0,1) # 2D list 索引
```

```
Out[84]: 2
```




实现二维列表类

- 二维列表
 - 二维列表的访问，索引和获取

```
In [85]: customList.myIndex(0,1) # 2D list 元素索引
```

```
Out[85]: 2
```

```
In [86]: customList.myIndex(0, None) # 2D list 行索引
```

```
Out[86]: [1, 2, 3]
```

```
In [87]: customList.myIndex(None, 1) # 2D list 列索引
```

```
Out[87]: [2, 5]
```



实现二维列表类

- 二维列表
 - 二维列表的元素增加与删除

```
In [99]: customList.myAppendCol([7,8]) # 2D list 列添加
```

```
In [100]: customList.getMy2DList()
```

```
Out[100]: [[1, 2, 3, 7], [4, 5, 6, 8]]
```

```
In [101]: customList.mySize()
```

```
Out[101]: (2, 4)
```

```
In [102]: customList.myDeleteCol(2) # 2D list 列删除
```

```
In [103]: customList.getMy2DList()
```

```
Out[103]: [[1, 2, 7], [4, 5, 8]]
```



回顾



人工智能与
Python程序设计

- 1. 获取对象信息
- 2. 实例属性和类属性
- 3. 实现二维列表类



谢谢！